



Srednja šola tehniških strok Šiška

4. predmet poklicne mature

Tehnik računalništva PTI

Izdelava 2D igre s C# in Unity

Avtor: Rok Rihar

Mentor: Tom Kamin, dipl. inž.

Ljubljana, junij 2023

Naziv programa: **Tehnik računalništva PTI**
Naziv srednje strokovne izobrazbe: **Tehnik računalništva**

Sklep o odobritvi teme in naslova IZDELKA ALI STORITVE za 4. predmet poklicne mature

Kandidat **Rihar Rok**, dijak **TR/5.A** oddelka opravlja po sklepu šolske maturitetne komisije izdelek oziroma storitev v obliki projektne dela pod zaporedno številko **TR-12/2023** (samostojna naloga) z naslovom:

Izdelava 2D igre s C# in Unity.
--

Mentor pri izdelavi projektne naloge je Tom Kamin.

Kandidat je dolžan projektno nalogo oddati v enem natisnjenem izvodu ter v elektronski obliki (Word). Nalogo je treba oddati tajniku šolske maturitetne komisije do petka, 2. 6. 2023, za spomladanski rok poklicne mature oziroma najkasneje 10 dni pred začetkom poznejših izpitnih rokov.

Kandidat mora obvezno opraviti najmanj dve konzultaciji pri mentorju. Brez opravljenih konzultacij ne sme pristopiti k zagovoru 4. predmeta poklicne mature. O konzultacijah se vodi zapisnik.

Kandidat je seznanjen z določili izpitnega reda in s posledicami kršenja Zakona o maturi in Pravilnika o poklicni maturi.

V Ljubljani, 1. junij 2023

Darinka Martinčič Zalokar,
Predsednica ŠMK SŠTS Šiška

1. Povzetek

Za zaključno nalogo sem ustvaril 2D igro v programu Unity. Igra je sestavljena iz štiri scen: glavni meni, posnetek, Level1 in Level2. V glavnem meniju lahko vstopimo v igro ali izven nje. V sceni Level1 sem ustvaril objekte: igralca, ki lahko strela ognjene kroglice in lahko skače po stenah, sovražnikov na blizu in na daleč, pasti in zemljišča po katerem se igralec premika (Večina objektov ima animacije in zvoke). Scena Level2 je pa zaključna bitka s glavnim sovražnikom, ki igralca zasleduje. Za izdelavo igre sem uporabil programski jezik C#.

1.1 Povzetek v angleščini

For my final assignment, I created a 2D game in Unity. The game consists of four scenes: Main Menu, Cutscene, Level1, and Level2. In the main menu, you can enter or exit the game. In scene Level1, I created a player who can shoot fireballs, jump on walls, encounter enemies both melee and ranged, encounter traps, and a terrain which player can walk on (most of them also have animations and sounds). Scene Level2 is the final battle with an main enemy who chases the player. The programing language that i used for the game is C#.

Kazalo vsebine

1.	Povzetek	1
1.1	Povzetek v angleščini	1
2.	Uvod	5
3.	Načrtovanje video igre	6
4.	Igralec	7
4.1	Fizike igralca + zaznavanje trkov in animacije	7
4.2	Skripta za premikanja igralca (PlayerMovement.)	7
4.3	Skripta za življenjske točke	10
4.4	Skripta za napad igralca	12
5.	Projektili	14
5.1	Ognjena krogla igralca	14
5.2	Ognjena krogla sovražnika in puščica za pasti	16
6.	Zbirateljski predmeti	18
6.1	Kontrolna točka	18
6.2	Srček	19
7.	Glavna kamera	20
8.	Pasti	21
8.1	Konice	21
8.2	Ognjena past	21
8.3	Past s puščicam	23
8.4	Kamnita glava	24
9.	Sovražniki	27
9.1	Sovražnik na blizu	27
9.2	Sovražnik na daleč	29
9.3	Glavni sovražnik (Šef (Boss))	31
10.	Zemljišče scen Level1 in Level2	36
11.	Meniji	37
11.1	Glavni meni	37
11.2	Meni konec igre in meni za premor	38
12.	Posnetek	41
13.	Zaključek	42
14.	Viri in literatura	43
15.	Zahvala mentorju	43
16.	Izjava o avtorstvu	45

Kazalo slik

Slika 1: Objekt igralca in nastavljene komponente	7
Slika 2: Vrednosti spremenljivk PlayerMovement.cs	8
Slika 3: Delovanje metod PlayerMovement.cs 1	9
Slika 4: Delovanje metod PlayerMovement.cs 2	9
Slika 5: Vrednosti spremenljivk Health.cs	10
Slika 6: Delovanje metod Health.cs 1	11
Slika 7: Delovanje metod Health.cs 2	12
Slika 8: Vrednosti spremenljivk PlayerAttack.cs	12
Slika 9: Delovanje metod PlayerAttack.cs	13
Slika 10: Prikaz objektov ognjenih krogel z objektom igralca	14
Slika 11: Zgled ognjenih krogel igralca	14
Slika 12: Delovanje metod Projectiles.cs	15
Slika 13: Zgled ognjene krogle sovražnika	16
Slika 14: Zgled projektila puščice	16
Slika 15: Delovanje metod EnemyProjectiles.cs	17
Slika 16: Zgled kontrolne točke	18
Slika 17: Delovanje metod PlayerRespawn.cs	19
Slika 18: Zgled srčka	19
Slika 19: Deklaracije spremenljivk in delovanje metod HealthCollectable.cs	19
Slika 20: Zgled glavne kamere v urejevalniku	20
Slika 21: Deklaracije spremenljivk in delovanje metod CameraController.cs	20
Slika 22: Zgled pasti konice	21
Slika 23: Deklaracije spremenljivk in delovanje metod EnemyDamage.cs	21
Slika 24: Zgled ognjene pasti	21
Slika 25: Delovanje metod Enemy_sideways.cs	22
Slika 26: Zgled pasti s puščicami	23
Slika 27: Prikaz objektov v pasti s puščicami	23
Slika 28: Delovanje metod Arrowtrap.cs	24
Slika 29: Zgled pasti kamnite glave	24
Slika 30: Vrednosti spremenljivk Rockhead.cs	25
Slika 31: Delovanje metod Rockhead.cs	26
Slika 32: Zgled sovražnika na blizu	27
Slika 33: Vrednosti spremenljivk MeleeEnemy.cs	27
Slika 34: Delovanje metod MeleeEnemy.cs	28
Slika 35: Zgled sovražnika na daleč	29
Slika 36: Vrednosti spremenljivk RangedEnemy.cs	30
Slika 37: Delovanje metod RangedEnemy.cs	31
Slika 38: Zgled glavnega sovražnika	31
Slika 39: Vrednosti spremenljivk Boss.cs	32
Slika 40: Skripta Boss.cs	32
Slika 41: Vrednosti spremenljivk BossHealth.cs	32
Slika 42: Deklaracija spremenljivk BossHealth.cs	32
Slika 43: Delovanje metod BossHealth.cs	33
Slika 44: Vrednosti spremenljivk BossWeapon.cs	33

Slika 45: Skripta BossWeapon.cs	33
Slika 46: Prikaz dodeljene skripte Boss_Run.cs na Run animacijo	34
Slika 47: Skripta Boss_Run.cs	34
Slika 48: Delovanje metod Boss_Enrage.cs	35
Slika 49: Zgled objekta tal	36
Slika 50: Zgled objekta stropa	36
Slika 51: Zgled objekta stene	36
Slika 52: Prikaz glavnega menija v urejevalniku	37
Slika 53: Delovanje metod MainMenu.cs	37
Slika 54: Zgled menija konec igre	38
Slika 55: Zgled menija za premor	38
Slika 56: Delovanje metod UIManager.cs	39
Slika 57: Delovanje metod SelectionArrow.cs	40
Slika 58: Zgled Posnetka in orodja Timeline v urejevalniku	41
Slika 59: Delovanje metod Cutscene.cs	41

2. Uvod

Tema obravnavane naloge je 2D video igra z 8-bit zgledom. Igra se imenuje »Emberius: The Blaze Guardian« video igri sem dodal tudi kratko zgodbo. V igri igrate ognjenega čarovnika, ki preganja gobline in hudobne čarovnike, ki so vpadli na čarovnikovo deželo imenovano Pyrovia. Prevod zgodbe:

»V mističnem kraljestvu Pyrovia, skrito med mogočnimi gorami, je živel ognjeni čarovnik imenovan Emberius. S svojo ognjeno rdečo haljo in palico, ki je švigala s plameni, je Emberius brez primere obvladal element ognja.

Stoletja je Emberius varoval svojo deželo pred kakršnimi koli grožnjami, ki so upale izzvati njegovo oblast. Toda usodnega dne je bila mirnost Pyovie prekinjena, ko je nad deželo prihrumela vojska zlobnih goblinov in temnih čarovnikov. Hrepeneli so po Emberiusovih močeh, da bi jih prisvojili zase in širili kaos po celotnem kraljestvu.

Ko so se napadalci bližali mejam in za seboj puščali zgorela opustošenja, Emberius ni mogel stati križem rok...«

Za video igro sem si izbral program Unity saj je enostaven za uporabo, ima intuitiven grafični vmesnik, hiter, lahek in brezplačen dostop do virov (zvokov, slik, animacij, fontov...), na voljo pa je tudi veliko vadnic za uporabo programa. Za uporabo Unityja sem se tudi potreboval naučiti programski C# in orodja, ki jih ponuja Unity.

Kot sem že omenil igra je sestavljena iz dveh glavnih delov – Level1 in Level2. V prvem delu (Level1) igralec se mora izogibati sovražnikom in pastem, ter priti do cilja, kjer se začne Level2. V drugem delu (Level2) pa mora igralec premagati glavnega sovražnika, ki ima drugačno obnašanje kot drugi sovražniki. V primeru, da igralec premaga glavnega sovražnika se igra konča.

3. Načrtovanje video igre

Pred začetkom razvoja videoigre sem ustvaril koncept igre. Moja prvotna ideja je bila ustvariti 2D samurai igro z 8-bit izgledom in v slogu hack and slash, vendar zaradi pomanjkanja znanja o programiranju nisem mogel uresničiti te vizije.

Sredstva za igro sem pridobil s nakupom, saj je ustvarjanje lastnih virov zelo časovno zahtevno. Sredstva sem kupil v trgovini sredstev Unity (Unity Asset Store). Nekatere slike sem pridobil tudi brezplačno iz iste trgovine.

Za zvočne elemente igre sem uporabil vire s spletnega mesta YouTube, ki sem jih našel v video vadnicah in glasbenih videih. Avtorji teh virov so dovolili uporabo glasbe pri razvoju iger.

Proces ustvarjanja igre je potekal po naslednjem postopku:

1. Začel sem z obnašanjem igralca, vključno s premikanjem, animacijami, skakanjem in napadi, ter obnašanjem projektilov igralca.
2. Razvil sem logiko kamere, ki sledi igralcu.
3. Implementiral sem logiko življenjskih točk igralca, ki sem jo kasneje uporabil tudi za sovražnike.
4. Dodal sem pasti v igro, vključno z njihovim obnašanjem in animacijami.
5. Razvil sem sovražnike v igri, vključno z njihovim obnašanjem in animacijami.
6. Dodal sem zvočne elemente v igro.
7. Implementiral sem logiko za ponovno rojstvo igralca po smrti.
8. Ustvaril sem zemljišče za prvi nivo igre (Level1).
9. Razvil sem glavni meni igre.
10. Dodal sem meni za konec igre in meni za premor med igro.
11. Ustvaril sem zemljišče za drugi nivo igre (Level2).
12. Implementiral sem logiko za glavnega sovražnika.
13. Zaključil sem igro in dodal logiko za konec igre.
14. Vključil sem vmesne posnetke med glavnim menijem in Level1 ter dodal zgodbo igre.
15. Izvedel sem odpravljanje napak in dokončal razvoj igre.

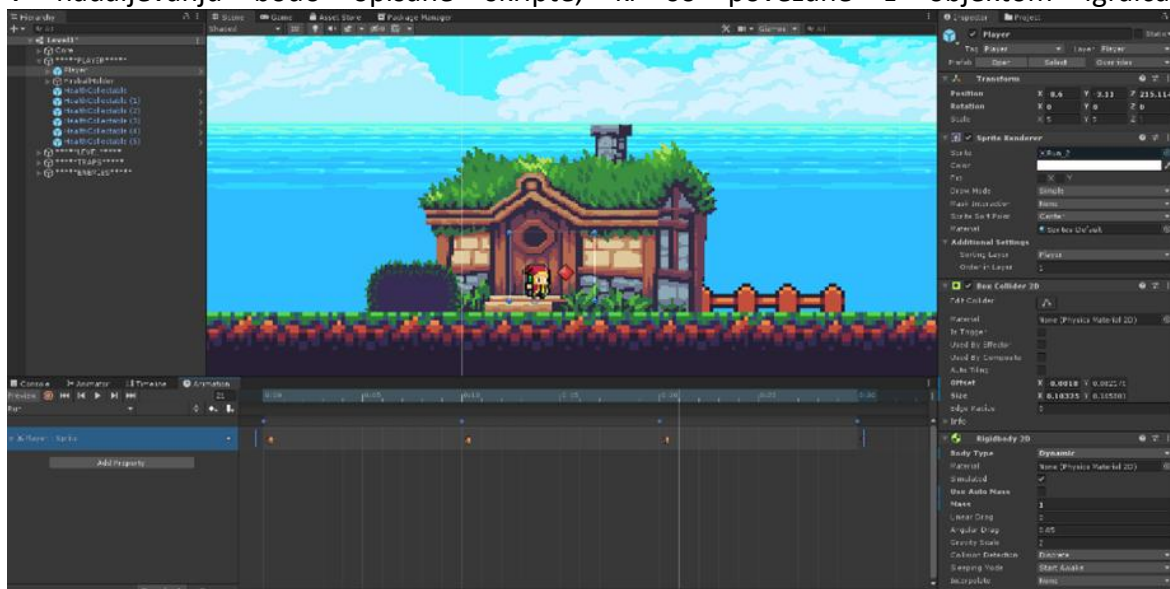
4. Igralec

4.1 Fizike igralca + zaznavanje trkov in animacije

Objekt igralca (Player) je opremljen z naslednjimi komponentami: **BoxCollider2D** in **Rigidbody2D**. BoxCollider2D je pravokotni kolider, ki omogoča zaznavanje trkov med igralcem in drugimi objekti v 2D igri. Rigidbody2D pa se uporablja za simulacijo fizike gibanja objekta v 2D prostoru. S to komponento določamo maso, trenje, gravitacijo in druge fizikalne lastnosti objekta.

Objekt igralca vsebuje tudi animacije za tek (run), napad (attack), skok (jump) in druge akcije. Za upravljanje s temi animacijami in njihovimi prehodi se uporablja Animator komponenta. Animator omogoča upravljanje in predvajanje animacij za objekte, elemente grafičnega vmesnika, posnetke in podobno. V Animator komponenti določujemo tranzicije med animacijami ter različne parametre, ki vplivajo na njihovo predvajanje. Te parametre lahko nato sklicujemo in uporabljamo v kodi.

V nadaljevanju bodo opisane skripte, ki so povezane z objektom igralca.



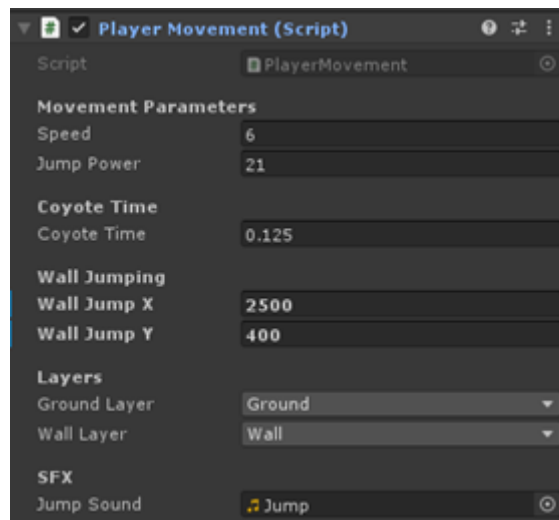
Slika 1: Objekt igralca in nastavljene komponente

4.2 Skripta za premikanja igralca (PlayerMovement.)

PlayerMovement je skripta, ki je odgovorna za premikanje igralca in obnašanje objektov, kot so tla ali zidovi. Vključuje naslednje parametre in metode:

- **Speed (float):** Določa hitrost premikanja igralca.
- **jumpPower (float):** Določa moč skoka igralca.
- **coyoteTime (float):** Določa čas, v katerem lahko igralec še skoči po tem, ko zapusti platformo.
- **coyoteCounter (float):** Meri preostali čas od začetka teka.
- **wallJumpX (float):** Določa horizontalno smer skoka ob zidu.
- **wallJumpY (float):** Določa vertikalno smer skoka ob zidu.
- **groundLayer (LayerMask):** Določa plasti za tla.

- **wallLayer (LayerMask)**: Določa plasti za zidove.
- **jumpSound (AudioClip)**: Zvočni posnetek za skok.



Slika 2: Vrednosti spremenljivk PlayerMovement.cs

Metode:

- **Awake()**: Se sproži ob zagonu skripte in pridobi reference na komponente **Rigidbody2D**, **Animator** in **BoxCollider2D** iz objekta.
- **Update()**: Se sproži v vsakem posameznem kadru. Prevzame vhod igralca, obrne igralca, nastavi parametre za animacije, preveri skoke, upravlja s premikanjem in zaznava stik s tlemi in zidovi.
- **Jump()**: Izvede skok igralca glede na trenutne pogoje, kot so prisotnost tla ali zidu. Uporablja tudi zvočni učinek skoka.
- **isGrounded()**: RaycastHit2D izstreli žarek in preveri če se je igralec dotaknil objekta v »groundLayer« plasti.
- **onWall()**: Preveri, ali je igralec na zidu s pomočjo **BoxCast** preverjanja stika s plasti zidov.
- **canAttack()**: Preveri, ali lahko igralec izvede napad glede na pogoje, kot so horizontalni vhod, prisotnost tal in odsotnost zidu.
- **WallJump()**: Izvede skok ob zidu z uporabo spremembe gravitacije in sile.

```

private void Awake()
{
    body = GetComponent<Rigidbody2D>();
    anim = GetComponent<Animator>();
    boxCollider = GetComponent<BoxCollider2D>();
}

@ Unity Message | 0 references
private void Update()
{
    horizontalInput = Input.GetAxis("Horizontal");

    if (horizontalInput > 0.01f)
        transform.localScale = new Vector3(5, 5, 1);
    else if (horizontalInput < -0.01f)
        transform.localScale = new Vector3(-5, 5, 1);

    anim.SetBool("run", horizontalInput != 0);
    anim.SetBool("grounded", isGrounded());

    if (Input.GetKeyDown(KeyCode.Space))
    {
        Jump();
    }

    if (Input.GetKeyDown(KeyCode.Space) && body.velocity.y > 0)
    {
        body.velocity = new Vector2(body.velocity.x, body.velocity.y / 2);
    }

    if (onWall())
    {
        body.gravityScale = 0;
        body.velocity = Vector2.zero;
    }
    else
    {
        body.gravityScale = 2;
        body.velocity = new Vector2(horizontalInput * speed, body.velocity.y);

        if (isGrounded())
        {
            coyoteCounter = coyoteTime;
        }
        else
        {
            coyoteCounter -= Time.deltaTime;
        }
    }
}

```

Slika 3: Delovanje metod PlayerMovement.cs 1

```

private void Jump()
{
    if (coyoteCounter < 0 && !onWall()) return;

    SoundManager.Instance.PlaySound(jumpSound);

    if (onWall())
        WallJump();
    else
    {
        if (isGrounded())
            body.velocity = new Vector2(body.velocity.x, jumpPower);
        else
        {
            if (coyoteCounter > 0)
            {
                body.velocity = new Vector2(body.velocity.x, jumpPower);
            }
        }
        coyoteCounter = 0;
    }
}

@ references
private bool isGrounded()
{
    RaycastHit2D raycastHit = Physics2D.BoxCast(boxCollider.bounds.center, boxCollider.bounds.size, 0, Vector2.down, 0.1f, groundLayer);
    return raycastHit.collider != null;
}

@ references
private bool onWall()
{
    RaycastHit2D raycastHit = Physics2D.BoxCast(boxCollider.bounds.center, boxCollider.bounds.size, 0, new Vector2(transform.localScale.x, 0), 0.1f, wallLayer);
    return raycastHit.collider != null;
}

@ reference
public bool canAttack()
{
    return horizontalInput == 0 && isGrounded() && !onWall();
}

@ reference
private void WallJump()
{
    body.gravityScale = -2;
    body.AddForce(new Vector2(-Mathf.Sign(transform.localScale.x) * wallJumpX, wallJumpY));
    wallJumpCooldown = 0;
}

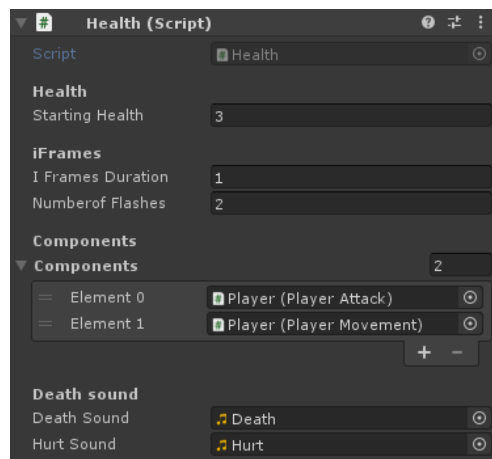
```

Slika 4: Delovanje metod PlayerMovement.cs 2

4.3 Skripta za življenjske točke

Skripta **Health.cs** upravlja življenjske točke objekta. Skripta se uporablja pri igralcu in sovražnikih. Vključuje funkcionalnosti za odzemanje in dodajanje življenjskih točk, ponovno oživljanje in obvladovanje okvirjev neranljivosti (iFrames).

- **startingHealth (float)**: predstavlja začetno vrednost zdravja igralca.
- **currentHealth (float)**: hrani trenutno vrednost življenjskih točk igralca. Oznaka **public** omogoča dostop do te spremenljivke iz drugih razredov, medtem ko **private set** omejuje nastavljanje vrednosti le znotraj razreda.
- **anim (Animator)**: predstavlja komponento Animator, ki omogoča upravljanje animacij za objekt igralca.
- **dead (bool)**: je logična spremenljivka, ki označuje, ali je igralec mrtev ali ne.
- **iFramesDuration (float)**: določa trajanje neranjenosti igralca po prejetem udarcu.
- **numberOfFlashes (int)**: predstavlja število utripov igralca med neranjenost.
- **spriteRend (SpriteRenderer)**: predstavlja komponento **SpriteRenderer**, ki omogoča upravljanje prikaza in barve igralčevega spritea.
- **components (Behaviour[])**: je tabela komponent tipa **Behaviour**, ki omogoča vklop in izklop določenih komponent igralca.
- **deathSound** in **hurtSound** sta avdio posnetka, ki se uporabljata za predvajanje zvoka ob smrti in zadetku igralca.



Slika 5: Vrednosti spremenljivk Health.cs

Metode:

- **TakeDamage(float _damage)**: Uporablja se za odštevanje določene količine življenjskih točk (damage) od trenutnih življenjskih točk. Če je življenjskih točk še vedno večje od 0, se predvaja animacija "hurt" in se začne neranljivost. Predvaja se tudi zvok poškodbe.
- **AddHealth(float _value)**: Uporablja se za dodajanje določene vrednosti življenjskih točk.
- **Respawn()**: Ponovno oživi objekt. Nastavi življenjske točke na začetno vrednost, ponastavi animacije in vklopi vse komponente.

```

public void TakeDamage(float _damage)
{
    currentHealth = Mathf.Clamp(currentHealth - _damage, 0, startingHealth);

    if (currentHealth > 0)
    {
        anim.SetTrigger("hurt");
        StartCoroutine(Invulnerability());
        SoundManager.Instance.PlaySound(hurtSound);
    }
    else
    {
        if (!dead)
        {
            foreach (Behaviour component in components)
            {
                component.enabled = false;
            }

            anim.SetBool("grounded", true);
            anim.SetTrigger("die");

            dead = true;
            SoundManager.Instance.PlaySound(deathSound);
        }
    }
}

2 references
public void AddHealth(float _value)
{
    currentHealth = Mathf.Clamp(currentHealth + _value, 0, startingHealth);
}

1 reference
public void Respawn()
{
    dead = false;
    AddHealth(startingHealth);
    anim.ResetTrigger("die");
    anim.Play("Idle");
    StartCoroutine(Invulnerability());

    foreach (Behaviour component in components)
    {
        component.enabled = true;
    }
}

```

Slika 6: Delovanje metod Health.cs 1

- **Awake():** Izvaja se ob prebujanju objekta. Nastavi začetne življenjske točke, animator in **SpriteRenderer** komponento.
- **Invulnerability():** Rutina, ki omogoča invulnerability za določen čas. Ignorira kolizijo na določenih plasteh in spreminja barvo objekta med določenim številom bliskov.
- **Deactivate():** Onemogoči objekt tako, da nastavi njegov **gameObject** na neaktivnega, po tem, ko igralec ali sovražnik umre.

```

private void Awake()
{
    currentHealth = startingHealth;
    anim = GetComponent<Animator>();
    spriteRend = GetComponent<SpriteRenderer>();
}
2 references
private IEnumerator Invulnerability()
{
    Physics2D.IgnoreLayerCollision(7, 8, true);
    for (int i = 0; i < numberOfFlashes; i++)
    {
        spriteRend.color = new Color(1, 0, 0, 0.5f);
        yield return new WaitForSeconds(iFramesDuration / (numberOfFlashes * 2));
        spriteRend.color = Color.white;
        yield return new WaitForSeconds(iFramesDuration / (numberOfFlashes * 2));
    }
    Physics2D.IgnoreLayerCollision(7, 8, false);
}
0 references
private void Deactivate()
{
    gameObject.SetActive(false);
}

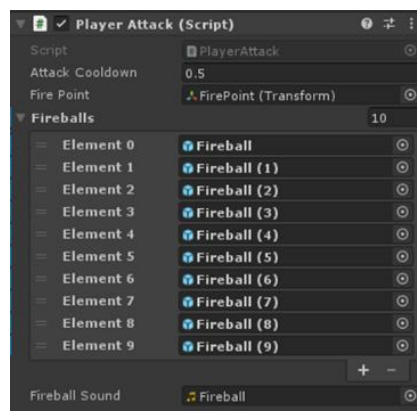
```

Slika 7: Delovanje metod Health.cs 2

4.4 Skripta za napad igralca

Skripta **PlayerAttack.cs** predstavlja vedenje napada igralca. Omogoča mu izvajanje napada z ognjenimi krogli:

- **attackCooldown (float)**: Določa časovni okvir med posameznimi napadi igralca.
- **firePoint (Transform)**: Referenca na **Transform** komponento, ki določa točko, iz katere se izstrelijo ognjene krogle.
- **Fireballs (GameObject[])**: Polje objektov (**GameObject**), ki predstavljajo ognjene krogle, ki jih lahko igralec izstrelji.
- **fireballSound (AudioClip)**: Zvok, ki se predvaja ob izstrelitvi ognjene krogle.
- **anim (Animator)**: Referenca na komponento Animator igralca, ki omogoča predvajanje animacij.
- **playerMovement (PlayerMovement)**: Referenca na komponento **PlayerMovement**, ki upravlja igralčevo premikanje.
- **cooldownTimer (float)**: Meri čas, ki je potekel od zadnjega napada igralca. Nastavljen je na neskončno, da lahko igralec takoj napade.



Slika 8: Vrednosti spremenljivk PlayerAttack.cs

Metode:

- **Awake():** Vzpostavi referenco na komponento **Animator** in **PlayerMovement** ob zagonu skripte.
- **Update():** Preverja, ali je bil pritisnjen levi gumb miške in ali je časovnik ohlajanja pretekel ter ali igralec lahko napade (preverjeno s pomočjo metode **canAttack()** iz **PlayerMovement**). Če so izpolnjeni pogoji, izvede napad.
- **Attack():** Predvaja zvok ognjene krogle preko **SoundManager**-ja. Sproži animacijo napada na komponenti **Animator**. Ponastavi časovnik ohlajanja na 0. Uporabi objektno baziranje (object pooling), da postavi ognjeno kroglo na ustrezno pozicijo. Nastavi smer gibanja ognjene krogle glede na smer, v katero je obrnjen igralec.
- **FindFireball():** Preišče seznam ognjenih krogel (**fireballs**) in vrne indeks prve neaktivne krogle. S tem omogoča recikliranje ognjenih krogel, namesto ustvarjanja novih ob vsakem napadu.

```
private void Awake()
{
    anim = GetComponent<Animator>();
    playerMovement = GetComponent<PlayerMovement>();
}

Unity Message | 0 references
private void Update()
{
    if (Input.GetMouseButton(0) && cooldownTimer > attackCooldown && playerMovement.canAttack())
        Attack();

    cooldownTimer += Time.deltaTime;
}

1 reference
private void Attack()
{
    SoundManager.instance.PlaySound(fireballSound);
    anim.SetTrigger("attack");
    cooldownTimer = 0;

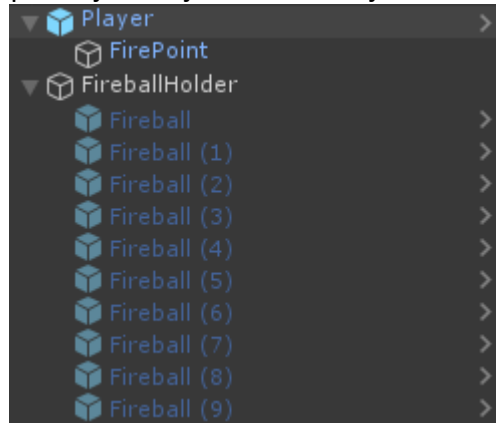
    //object pooling
    fireballs[FineFireball()].transform.position = firePoint.position;
    fireballs[FineFireball()].GetComponent<Projectile>().SetDirection(Mathf.Sign(transform.localScale.x));
}

2 references
private int FineFireball()
{
    for (int i = 0; i < fireballs.Length; i++)
    {
        if (!fireballs[i].activeInHierarchy)
            return i;
    }
    return 0;
}
```

Slika 9: Delovanje metod PlayerAttack.cs

5. Projektili

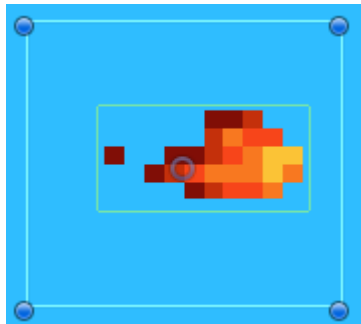
V igri sem implementiral tri različne projekte objekte. Vsi objekti, ki lahko izstreljujejo projektil, imajo določeno točko, iz katere se izstreljujejo, imenovano "firepoint". Ognjene krogle imajo poleg tega še določene animacije za premikanje skozi zrak in za eksplozijo ob trku. Igralec in sovražnik uporabljata objektno baziranje za izstrelitev ognjenih krogel.



Slika 10: Prikaz objektov ognjenih krogel z objektom igralca

Objektno baziranje je mehanizem, pri katerem ustvarimo več objektov ognjenih krogel in namesto ustvarjanja novega objekta ob vsakem strelu, preprosto aktiviramo in deaktiviramo že obstoječe objekte ognjenih krogel. S tem pristopom dosežemo boljšo učinkovitost programa, saj se izognemo nepotrebnemu ustvarjanju in uničevanju objektov ter izboljšamo delovanje igre.

5.1 Ognjena krogla igralca



Slika 11: Zgled ognjenih krogel igralca

Skripta Projectiles.cs:

- **speed (float):** Hitrost premikanja izstrelka. (V igri je nastavljena na 10)
- **direction (float):** Smer, v katero se premika izstrelka.
- **hit (bool):** Določa, ali je izstrelka zadel nekaj.
- **lifetime (float):** Čas, ki je izstrelka preživel aktiven.
- **boxCollider (BoxCollider2D):** Komponenta BoxCollider2D, pripeta na izstrelku.
- **anim (Animator):** Komponenta Animator, pripeta na izstrelku.

Metode:

- **Update():** Metoda, ki se kliče enkrat na vsakem posodobitvenem koraku. Posodablja položaj izstrelka glede na hitrost in smer. Prav tako preverja, ali je preteklo dovolj časa od izstrelka, da ga deaktivira.
- **OnTriggerEnter2D(Collider2D collision):** Metoda, ki se kliče ob trku izstrelka s katerim koli drugim trkom. Obvladuje logiko trka izstrelka z sovražnikom, kot je onemogočanje trka, predvajanje animacije eksplozije ter odvzemanje življenjskih točk sovražnikom.
- **SetDirection(float _direction):** Nastavi smer izstrelka in ga aktivira. Prav tako preklopi usmerjenost izstrelkove slike glede na smer.
- **Deactivate():** Deaktivira izstrelka, ga neaktivnega v igri.

```
private void Update()
{
    if (hit) return;
    float movementSpeed = speed * Time.deltaTime * direction;
    transform.Translate(movementSpeed, 0, 0);

    lifetime += Time.deltaTime;
    if (lifetime > 8) gameObject.SetActive(false);
}

@ Unity Message | 0 references
private void OnTriggerEnter2D(Collider2D collision)
{
    hit = true;
    boxCollider.enabled = false;
    anim.SetTrigger("explode");

    if (collision.tag == "Enemy")
    {
        collision.GetComponent<Health>().TakeDamage(1);
    }
}

1 reference
public void SetDirection(float _direction)
{
    lifetime = 0;
    direction = _direction;
    gameObject.SetActive(true);
    hit = false;
    boxCollider.enabled = true;

    float localScaleX = transform.localScale.x;
    if (Mathf.Sign(localScaleX) != _direction)
        localScaleX = -localScaleX;

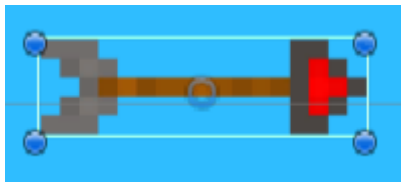
    transform.localScale = new Vector3(localScaleX, transform.localScale.y, transform.localScale.z);
}

0 references
private void Deactivate()
{
    gameObject.SetActive(false);
}
```

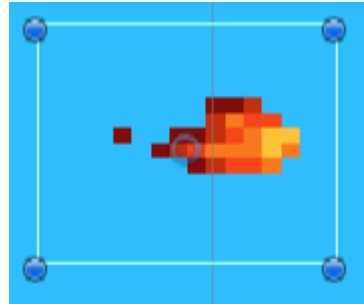
Slika 12: Delovanje metod Projectiles.cs

5.2 Ognjena krogla sovražnika in puščica za pasti

Skripta za ognjeno kroglo sovražnika in puščico za pasti sta enaki, saj imata oba objekta enak namen - odvzem življenjskih točk igralcu. Kljub temu pa obstajajo nekatere razlike med njima. Ognjena krogla ima dodatne animacije, medtem ko je puščica zgolj statična slika. Poleg tega puščice potujejo hitreje in imajo manjšo zakasnitev med streljanjem v primerjavi z ognjenimi krogli. Skripta, ki jo uporabljata obe vrsti projektilov, pa ima podobne lastnosti, kot skripta za ognjene krogle igralca.



Slika 14: Zgled projektila puščice



Slika 13: Zgled ognjene krogle sovražnika

Skripta EnemyProjectiles.cs:

- **speed (float):** Hitrost premikanja projektila. Določa, kako hitro se projektil premika v določeni smeri.
- **resetTime (float):** Čas, po katerem se projektil samodejno deaktivira, če ni trčil s katerim koli objektom.
- **lifetime (float):** Trenutna življenjska doba projektila. Uporablja se za sledenje času, ki je pretekel od aktivacije projektila.
- **anim (Animator):** Referenca na komponento **Animator**, ki upravlja animacije projektila. Ta referenca se uporablja za sprožitev animacije eksplozije projektila.
- **coll (BoxCollider2D):** Referenca na **BoxCollider2D**, ki predstavlja kolajder projektila. Ta kolajder se uporablja za zaznavanje trkov projektila s kolajderji drugih objektov.
- **hit (bool):** Zastavica, ki označuje, ali je projektil že trčil s katerim koli objektom. Ta zastavica preprečuje nadaljnje premikanje projektila po trku.

Metode:

- **ActivateProjectile():** Metoda omogoča aktivacijo projektila za uporabo. Nastavi **hit** na **false**, ponastavi življenjsko dobo **lifetime** ter omogoči kolizijo s kolajderji drugih objektov.
- **Update():** Metoda se izvaja vsakokratno posodobitev in premika projektil v določeni smeri. Hitrost premika je odvisna od **speed** in časa pretekle med dvema posodobitvama. Če življenjska doba **lifetime** preseže časovno omejitev **resetTime**, se projektil deaktivira.
- **OnTriggerEnter2D(Collider2D collision):** Metoda se sproži ob trčenju projektila s kolajderjem drugega objekta. Najprej izvede logiko starševske skripte za obdelavo trka. Nato onemogoči kolizijo (**coll.enabled = false**) ter preveri, ali je na voljo animacija **anim**. Če animacija obstaja, sproži animacijo "explode" in s tem prikaže eksplozijo projektila. V nasprotnem primeru se projektil deaktivira.
- **Deactivate():** Metoda deaktivira projektil tako, da ga skripta izklopi (**gameObject.SetActive(false)**).

```

0 references
public void ActivateProjectile()
{
    hit = false;
    lifetime = 0;
    gameObject.SetActive(true);
    coll.enabled = true;
}

@ Unity Message | 0 references
private void Update()
{
    if (hit) return;

    float movementSpeed = speed * Time.deltaTime;
    transform.Translate(movementSpeed, 0, 0);

    lifetime += Time.deltaTime;
    if (lifetime > resetTime)
        gameObject.SetActive(false);
}

@ Unity Message | 0 references
private void OnTriggerEnter2D(Collider2D collision)
{
    hit = true;
    base.OnTriggerEnter2D(collision);
    coll.enabled = false;

    if (anim != null)
        anim.SetTrigger("explode");
    else
        gameObject.SetActive(false);
}

0 references
private void Deactivate()
{
    gameObject.SetActive(false);
}

```

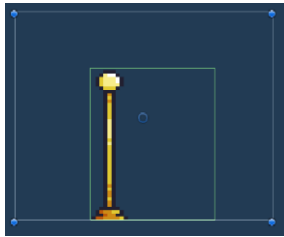
Slika 15: Delovanje metod EnemyProjectiles.cs

6. Zbirateljski predmeti

V igri sta 2 zbirateljska predmeta: kontrolna točka (checkpoint) in srček (health collectable).

6.1 Kontrolna točka

Kontrolna točka je objekt, ki omogoča igralcu, da doseže določeno točko v igri. Ko se igralec dotakne kontrolne točke, se predvaja animacija in nastavi se drstitvena točka, ki je povezana z igralcem. V primeru, da igralec izgubi vse življenjske točke, se bo ponovno pojavil na tej določeni točki. Pomembno je omeniti, da skripta, ki nadzoruje delovanje kontrolne točke, ni neposredno povezana z objektom same kontrolne točke, ampak je vezana na objekt igralca.



Slika 16: Zgled kontrolne točke

Skripta `PlayerRespawn.cs`:

- **checkpoint (AudioClip)**: Zvočni učinek, ki se predvaja ob dosegi točke ponovnega pojava.
- **currentCheckpoint (Transform)**: Sklic na trenutno točko ponovnega pojava, na katero se bo igralec premaknil ob ponovnem pojavljanju.
- **playerHealth (Health)**: Sklic na skripto **Health**, ki upravlja z življenjskimi točkami igralca.
- **uiManager (UIManager)**: Sklic na skripto **UIManager**, ki upravlja uporabniškim vmesnikom.

Metode:

- **CheckRespawn()**: Ta metoda preverja, ali obstaja trenutna točka ponovnega pojava. Če točka ne obstaja, se sproži konec igre (pokliče se metoda **GameOver()** iz **UIManager**). Če obstaja točka ponovnega pojava, se igralec ponovno pojavi, njegove življenjske točke se obnovijo, premakne se na točko ponovnega pojava.
- **OnTriggerEnter2D(Collider2D collision)**: Ta metoda se kliče ob trku z drugim objektom. Preverja, ali je objekt, s katerim je igralec trčil, označen kot "Checkpoint". Če je objekt točka ponovnega pojava, se nastavi kot trenutna točka ponovnega pojava, predvaja se zvočni učinek, in onemogoči se sprožilec trkov (**Collider2D**) objekta in animacija pojava (**appear**) se sproži na objektu.

```

@ Unity Message | 0 references
private void Awake()
{
    playerHealth = GetComponent<Health>();
    uiManager = FindObjectOfType<UIManager>();
}

0 references
public void CheckRespawn()
{
    if (currentCheckpoint == null)
    {
        uiManager.GameOver();

        return;
    }

    playerHealth.Respawn();
    transform.position = currentCheckpoint.position;
}

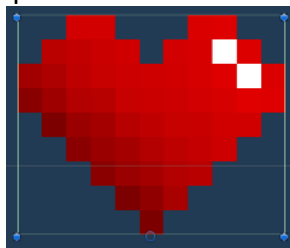
@ Unity Message | 0 references
private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.gameObject.tag == "Checkpoint")
    {
        currentCheckpoint = collision.transform;
        SoundManager.instance.PlaySound(checkpoint);
        collision.GetComponent<Collider2D>().enabled = false;
        collision.GetComponent<Animator>().SetTrigger("appear");
    }
}

```

Slika 17: Delovanje metod PlayerRespawn.cs

6.2 Srček

Srček deluje tako, da ko ga igralec pobere mu doda 1 življenjsko točko.



Slika 18: Zgled srčka

Skripta HealthCollectable.cs:

```

[SerializeField] private float healthValue;
[SerializeField] private AudioClip pickupSound;

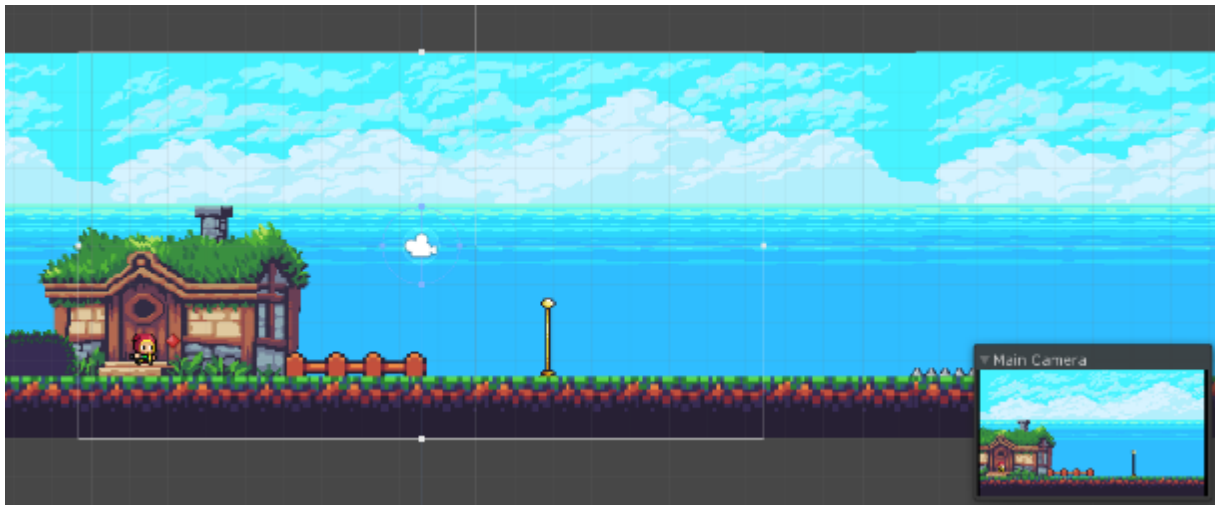
@ Unity Message | 0 references
private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.tag == "Player")
    {
        SoundManager.instance.PlaySound(pickupSound);
        collision.GetComponent<Health>().AddHealth(healthValue);
        gameObject.SetActive(false);
    }
}

```

Slika 19: Deklaracije spremenljivk in delovanje metod HealthCollectable.cs

7. Glavna kamera

V Level1 in Level2 scenah je kamera nastavljena, da igralcu sledi. V sceni glavnega menija in posnetka pa kamera miruje.



Slika 20: Zgled glavne kamere v urejevalniku

Skripta glavne kamere v Level1 in Level2 scenah CameraController.cs:

- **currentPosX (float):** Trenutna x-koordinata kamere med prehodom med sobami.
- **velocity (Vector3):** Vektor, ki predstavlja hitrost premikanja kamere. Na začetku je nastavljen na ničelni vektor (**Vector3.zero**).
- **player (Transform):** Referenca na transform komponento igralca, ki ga kamera sledi.

Metoda Update():

- Posodablja položaj kamere na trenutno x lokacijo igralca.

Metoda MoveToNewRoom(Transform _newRoom):

- Premakne kamero na novo sobo (novo transformacijo), tako da nastavi **currentPosX** na x-koordinato nove sobe.
- Ta metoda se uporablja za premikanje kamere med sobami v igri.

```
private float currentPosX;
private Vector3 velocity = Vector3.zero;

[SerializeField] private Transform player;

© Unity Message | 0 references
private void Update()
{
    transform.position = new Vector3(player.position.x, transform.position.y, transform.position.z);
}

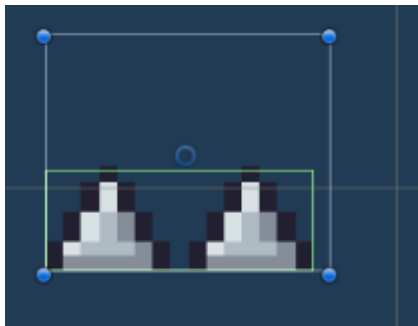
1 reference
public void MoveToNewRoom(Transform _newRoom)
{
    currentPosX = _newRoom.position.x;
}
```

Slika 21: Deklaracije spremenljivk in delovanje metod CameraController.cs

8. Pasti

V igri so 4 pasti, vsaka ima svoje posebne lastnosti in način delovanja. Pasti se uporabljajo zgolj v sceni Level1.

8.1 Konice



Slika 22: Zgled pasti konice

So preproste, če igralec stopi na past izgubi eno življenjsko točko.

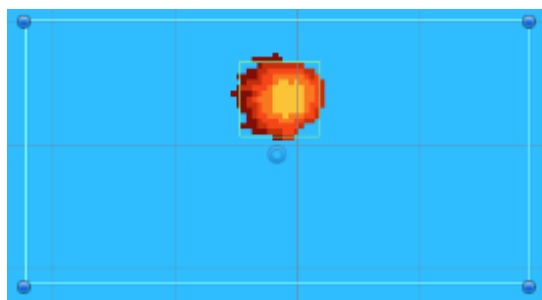
Skripta EnemyDamage.cs:

- **damage:** V igri je nastavljena na 1
- **OnTriggerEnter2D:**

```
[SerializeField] protected float damage;  
  
Unity Message | 2 references  
protected void OnTriggerEnter2D(Collider2D collision)  
{  
    if (collision.tag == "Player")  
        collision.GetComponent<Health>().TakeDamage(damage);  
}
```

Slika 23: Deklaracije spremenljivk in delovanje metod EnemyDamage.cs

8.2 Ognjena past



Slika 24: Zgled ognjene pasti

Past leti levo/desno z določeno distanco in hitrostjo. Objektu lahko nastavimo koliko življenjskih točk odvzame igralcu. Nastavljeni parametri ognjene pasti se razlikujejo glede na postavitev na sceni Level1.

Skripta Enemy_sideways.cs:

- **movementDistance (float):** določa razdaljo, ki jo bo sovražnik prehodil.
- **Speed (float):** določa hitrost premikanja sovražnika.
- **Damage (float):** določa koliko življenjskih točk vzame igralcu ob trku.
- **movingLeft (bool):** spremenljivka, ki določa smer premikanja sovražnika.
- **leftEdge in rightEdge(oba float):** določata levi in desni rob območja, v katerem se sovražnik premika.

Metode:

- **Awake:** se kliče ob zagonu skripte in inicializira vrednosti za **leftEdge** in **rightEdge**, ki temeljijo na trenutni poziciji sovražnika.
- **Update:** preverja trenutno smer premikanja sovražnika in ga premika v levo ali desno v skladu s speed in movementDistance. Ko sovražnik doseže levi ali desni rob območja, spremeni smer premikanja.
- **OnTriggerEnter2D:** preveri, če je trčil s koliderjem igralca in mu odvzame eno življenjsko točko

```
private void Awake()
{
    leftEdge = transform.position.x - movementDistance;
    rightEdge = transform.position.x + movementDistance;
}

// Unity Message | 0 references
private void Update()
{
    if (movingLeft)
    {
        if (transform.position.x > leftEdge)
        {
            transform.position = new Vector3(transform.position.x - speed * Time.deltaTime, transform.position.y, transform.position.z);
        }
        else
        {
            movingLeft = false;
        }
    }
    else
    {
        if (transform.position.x < rightEdge)
        {
            transform.position = new Vector3(transform.position.x + speed * Time.deltaTime, transform.position.y, transform.position.z);
        }
        else
        {
            movingLeft = true;
        }
    }
}

// Unity Message | 0 references
private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.tag == "Player")
    {
        collision.GetComponent<Health>().TakeDamage(damage);
    }
}
```

Slika 25: Delovanje metod Enemy_sideways.cs

8.3 Past s puščicam



Slika 27: Prikaz objektov v pasti s puščicami



Slika 26: Zgled pasti s puščicami

Past strelja puščice iz nastavljene točke. Skripta pasti tudi uporablja baziranje objektov. Puščice uporabljajo enako skripto, kot skripta ognjenih kroglic sovražnikov (skripto si lahko pogledate v naslednjem poglavju).

Skripta Arrowtrap.cs:

- **attackCooldown (float):** določa časovni interval med napadi pasti.
- **firePoint (Transform):** predstavlja transformacijo, kjer se izstreljajo puščice.
- **arrows (GameObject[]):** je polje objektov, ki predstavljajo puščice.
- **cooldownTimer (float)** meri čas, ki je pretekel od zadnjega napada pasti.

Metode:

- **Attack** se uporablja za izvedbo napada pasti. Ponastavi **cooldownTimer** na 0 in izbere neaktivno puščico iz polja **arrows**. Nastavi njeno pozicijo na **firePoint** in aktivira izstrelitev izstrelka s klicem metode **ActivateProjectile** iz komponente **EnemyProjectile** puščice.
- **FindArrow** se uporablja za iskanje neaktivne puščice v polju **arrows**. Uporablja zanko, ki preverja vsak element v polju. Vrne indeks prve neaktivne puščice. Če ne najde neaktivne puščice, vrne 0.
- **Update** Inkrementira **cooldownTimer** glede na pretečen čas. Preverja, ali je **cooldownTimer** dosegel ali presegel vrednost **attackCooldown**. Če je, izvede napad s klicem metode **Attack()**.

```

1 reference
private void Attack()
{
    cooldownTimer = 0;

    arrows[FindArrow()].transform.position = firePoint.position;
    arrows[FindArrow()].GetComponent<EnemyProjectile>().ActivateProjectile();
}

2 references
private int FindArrow()
{
    for (int i = 0; i < arrows.Length; i++)
    {
        if (!arrows[i].activeInHierarchy)
            return i;
    }
    return 0;
}

@ Unity Message | 0 references
private void Update()
{
    cooldownTimer += Time.deltaTime;

    if (cooldownTimer >= attackCooldown)
        Attack();
}

```

Slika 28: Delovanje metod Arrowtrap.cs

8.4 Kamnita glava



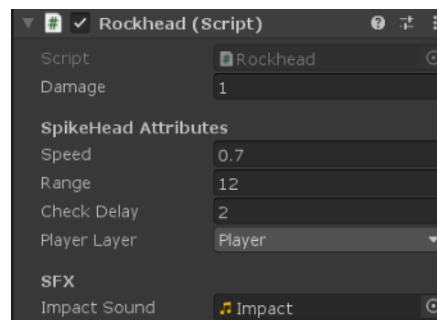
Slika 29: Zgled pasti kamnite glave

Kamnita glava (rockhead) je past, ki zazna, če se igralec nahaja pod/nad/levo/desno past in mu sledi.

Skripta Rockhead.cs:

- **Speed (float):** določa hitrost premikanja sovražnika.
- **range (float):** določa razdaljo, na kateri sovražnik zazna igralca.

- **checkDelay (float):** določa časovno zakasnitev med preverjanjem prisotnosti igralca.
- **playerLayer (LayerMask):** določa plast, na kateri se nahaja igralec.
- **Directions (Vector3[]):** je polje vektorjev, ki predstavljajo smeri, v katerih se preverja prisotnost igralca (nastavljen je na »new Vector3[4]«, da v seznam lahko shranimo samo 4 vrednosti).
- **destination (Vector3):** predstavlja ciljno smer, v katero se premika sovražnik med napadom.
- **checkTimer (float):** meri čas, ki je pretekel od zadnjega preverjanja prisotnosti igralca.
- **attacking (bool):** določa, ali je sovražnik v fazi napada.



Slika 30: Vrednosti spremenljivk Rockhead.cs

Metode:

- **Update:** preverja, če je kamnita glava v fazi napada (**attacking** je **true**), se sovražnik premika v ciljno smeri (**destination**) s hitrostjo **speed**. V nasprotnem primeru se povečuje **checkTimer**, ki meri čas od zadnjega preverjanja prisotnosti igralca. Ko **checkTimer** preseže **checkDelay**, se kliče metoda **CheckForPlayer()**.
- **CheckForPlayer:** preverja prisotnost igralca v vseh štirih smereh, tako da izvaja žarke preverjanja trčenja (**Raycast**) v vse smeri iz pozicije sovražnika. Če je trčenje zaznano na plasti igralca (**playerLayer**) in sovražnik ni že v fazi napada, se sovražnik preide v fazo napada. Nastavi se ciljna smer (**destination**) na smer, kjer je bil zaznan igralec, in **checkTimer** se ponastavi.
- **CalculateDirections:** izračuna smeri (**directions**) za preverjanje prisotnosti igralca. Uporablja se pomnoževanje vektorjev (**transform.right**, **transform.up**) s faktorjem **range**, da se določi razdalja v posamezni smeri.
- **Stop:** postavi ciljno smer (**destination**) na trenutno pozicijo sovražnika, kar preprečuje njegovo premikanje. Poleg tega postavi **attacking** na **false**, da označi konec napada.
- **OnTriggerEnter2D:** predvaja zvok **impactSound**, ko trči v nek drug kolider in nato se kliče metoda **OnTriggerEnter2D** iz starševske skripte (**EnemyDamage**), ki se uporablja za upravljanje logike poškodb sovražnikov. Nazadnje se kliče metoda **Stop()** za ustavitev sovražnika po trčenju.

```

@ Unity Message | 0 references
private void OnEnable()
{
    Stop();
}

@ Unity Message | 0 references
private void Update()
{
    if (attacking)
        transform.Translate(destination * Time.deltaTime * speed);
    else
    {
        checkTimer += Time.deltaTime;
        if (checkTimer > checkDelay)
            CheckForPlayer();
    }
}

1 reference
private void CheckForPlayer()
{
    CalculateDirections();

    for (int i = 0; i < directions.Length; i++)
    {
        Debug.DrawRay(transform.position, directions[i], Color.red);
        RaycastHit2D hit = Physics2D.Raycast(transform.position, directions[i], range, playerLayer);

        if (hit.collider != null && !attacking)
        {
            attacking = true;
            destination = directions[i];
            checkTimer = 0;
        }
    }
}

1 reference
private void CalculateDirections()
{
    directions[0] = transform.right * range;
    directions[1] = -transform.right * range;
    directions[2] = transform.up * range;
    directions[3] = -transform.up * range;
}

2 references
private void Stop()
{
    destination = transform.position;
    attacking = false;
}

@ Unity Message | 0 references
private void OnTriggerEnter2D(Collider2D collision)
{
    SoundManager.instance.PlaySound(impactSound);
    base.OnTriggerEnter2D(collision);
    Stop();
}

```

Slika 31: Deloavne metod Rockhead.cs

9. Sovražniki

V igri obstajajo 3 sovražniki vsi delujejo na različne načine. Uporabljajo **Health.cs** in **EnemyDamage.cs** skripto zato, da lahko igralcu odvzamejo življenjske točke (in obratno) ter za nastavitev število življenjskih točk.

9.1 Sovražnik na blizu



Slika 32: Zgled sovražnika na blizu

Sovražnik na blizu deluje tako, da ko pride igralec v njegov domet mu lahko odvzame eno življenjsko točko.

Skripta MeleeEnemy.cs:

- **attackCooldown (float):** Časovno obdobje med napadi sovražnika.
- **range (float):** Razdalja, na kateri sovražnik lahko zazna igralca in izvede napad.
- **damage (int):** Koliko življenjskih točk sovražnik odvzame.
- **colliderDistance (float):** Razdalja med sovražnikom in kolizijskim ovojem.
- **boxCollider (BoxCollider2D):** Referenca na komponento **BoxCollider2D**, ki predstavlja kolizijski ovoj sovražnika.
- **playerLayer (LayerMask):** Plast igralca, s katero sovražnik interagira.
- **cooldownTimer (float):** Časovnik, ki meri pretečeni čas od zadnjega napada sovražnika. Ima privzeto vrednost **Mathf.Infinity**, kar omogoča, da sovražnik takoj napade ob zagonu.
- **attackSound (AudioClip):** Zvok napada sovražnika.
- **anim (Animator):** Referenca na komponento **Animator**, ki se uporablja za predvajanje animacij sovražnika.
- **playerHealth (Health):** Referenca na skripto **Health**, ki predstavlja življenjske točke igralca.
- **enemyPatrol (EnemyPatrol):** Referenca na skripto **EnemyPatrol**, ki upravlja z gibanjem sovražnika med patroljiranjem.



Slika 33: Vrednosti spremenljivk *MeleeEnemy.cs*

Metode:

- **Awake():** Metoda se izvede ob zagonu skripte. Inicializira reference na komponente **Animator** in **EnemyPatrol**.
- **Update():** Metoda se kliče vsak posamezen korak. Povečuje časovnik **cooldownTimer** in preverja, ali je igralec v vidnem polju sovražnika. Če so izpolnjeni pogoji za napad, se sproži animacija napada. Poleg tega upravlja s stanjem patroljiranja sovražnika.
- **PlayerInSight():** Metoda preverja, ali je igralec v vidnem polju sovražnika. Uporablja **BoxCast** metode za zaznavanje kolizije z igralcem. Če je igralec zaznan, se pridobi referenca na njegove življenjske točke.
- **OnDrawGizmos():** Metoda se uporablja za risanje vizualnih pomoči v Unity Editorju. V tem primeru riše omejitveni ovoj (**WireCube**) okoli kolizijskega ovoja sovražnika.
- **DamagePlayer():** Metoda odvzame življenjske točke igralcu, če je igralec v vidnem polju sovražnika. Izvede tudi predvajanje zvoka napada.

```
private void Awake()
{
    anim = GetComponent<Animator>();
    enemyPatrol = GetComponentInParent<EnemyPatrol>();
}

// Unity Message / 0 references
private void Update()
{
    cooldownTimer += Time.deltaTime;
    if (PlayerInSight())
    {
        if (cooldownTimer >= attackCooldown && playerHealth.currentHealth > 0)
        {
            cooldownTimer = 0;
            anim.SetTrigger("meleeAttack");
        }
    }

    if (enemyPatrol != null)
    {
        enemyPatrol.enabled = !PlayerInSight();
    }
}

// references
private bool PlayerInSight()
{
    RaycastHit2D hit = Physics2D.BoxCast(boxCollider.bounds.center + transform.right * range * transform.localScale.x * colliderDistance,
    new Vector3(boxCollider.bounds.size.x * range, boxCollider.bounds.size.y, boxCollider.bounds.size.z),
    0, Vector2.left, 0, playerLayer);

    if (hit.collider != null)
    {
        playerHealth = hit.transform.GetComponent<Health>();
    }

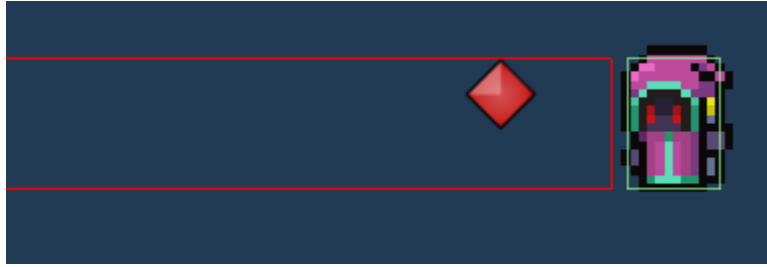
    return hit.collider != null;
}

// Unity Message / 0 references
private void OnDrawGizmos()
{
    Gizmos.color = Color.red;
    Gizmos.DrawWireCube(boxCollider.bounds.center + transform.right * range * transform.localScale.x * colliderDistance,
    new Vector3(boxCollider.bounds.size.x * range, boxCollider.bounds.size.y, boxCollider.bounds.size.z));
}

// references
private void DamagePlayer()
{
    if (PlayerInSight())
    {
        playerHealth.TakeDamage(damage);
        SoundManager.Instance.PlaySound(attackSound);
    }
}
```

Slika 34: Delovanje metod *MeleeEnemy.cs*

9.2 Sovražnik na daleč



Slika 35: Zgled sovražnika na daleč

Sovražnik na daleč deluje podobno kot sovražnik na blizu. Začne streljati ognjene krogle, ko je igralec v njegovem dometu, če se igralec zaleti v ognjeno kroglo mu odvzame eno življenjsko točko.

Skripta **RangedEnemy.cs**:

- **attackCooldown (float)**: Časovni interval med napadi sovražnika.
- **range (float)**: Razdalja, na kateri sovražnik lahko zazna igralca in izvede napad.
- **damage (int)**: Koliko življenjskih točk sovražnik odvzame.
- **firepoint (Transform)**: Transformacija, ki določa točko, kjer se ustvarjajo projektili.
- **fireballs (GameObject[])**: Polje igralnih objektov, ki predstavljajo projektili, ki jih sovražnik izstreljuje.
- **colliderDistance (float)**: Razdalja med sovražnikom in kolizijskim ovojem.
- **boxCollider (BoxCollider2D)**: Referenca na komponento **BoxCollider2D**, ki predstavlja kolizijski ovoj sovražnika.
- **playerLayer (LayerMask)**: Plast igralca, s katero sovražnik interagira.
- **cooldownTimer (float)**: Časovnik, ki meri pretečeni čas od zadnjega napada sovražnika. Ima privzeto vrednost **Mathf.Infinity**, kar omogoča, da sovražnik takoj napade ob zagonu.
- **fireballSound (AudioClip)**: Zvok, ki se predvaja ob izstrelitvi projektila.
- **anim (Animator)**: Referenca na komponento **Animator**, ki se uporablja za predvajanje animacij sovražnika.
- **playerHealth (Health)**: Referenca na skripto **Health**, ki predstavlja življenjske točke igralca.
- **enemyPatrol (EnemyPatrol)**: Referenca na skripto **EnemyPatrol**, ki upravlja z gibanjem sovražnika med patroljiranjem.



Slika 36: Vrednosti spremenljivk *RangedEnemy.cs*

Metode:

- **Awake():** Metoda se izvede ob zagonu skripte. Inicializira reference na komponente **Animator** in **EnemyPatrol**.
- **Update():** Metoda se kliče vsak posamezen korak. Povečuje časovnik **cooldownTimer** in preverja, ali je igralec v vidnem polju sovražnika. Če je izpolnjen pogoj za napad, se sproži animacija napada. Poleg tega upravlja s stanjem patroljiranja sovražnika.
- **RangedAttack():** Metoda se izvede ob napadu sovražnika na daljavo. Predvaja zvok projektila, ponastavi časovnik **cooldownTimer** in izstrelji projektil iz **firepoint** točke.
- **FindFireball():** Metoda poišče neaktivni projektil v polju **fireballs** in vrne njegov indeks. Uporablja se za pravilno dodelitev projektila pred izstrelitvijo.
- **PlayerInSight():** Metoda preverja, ali je igralec v vidnem polju sovražnika. Uporablja **BoxCast** metode za preverjanje, ali obstaja trk med kolizijskim ovojem sovražnika in igralcem. Metoda uporablja **Physics2D.BoxCast** funkcijo, ki pošilja žarek iz središča kolizijskega ovoja v določeni smeri in preverja, ali se ta žarek preseka s kolajderjem igralca. Vrne **true**, če je trk zaznan, sicer pa vrne **false**.
- **OnDrawGizmos():** Metoda se izvaja samo v Unity Editorju in riše vizualne elemente za pomoč pri razvoju. V tem primeru prikazuje ovoj kolizije sovražnika in njegovo vidno polje z uporabo **Gizmos.DrawWireCube** funkcije.


```

private void Awake()
{
    anim = GetComponent<Animator>();
    enemyPatrol = GetComponentInParent<EnemyPatrol>();
}

// Unity Message | 0 references
private void Update()
{
    cooldownTimer += Time.deltaTime;
    if (PlayerInSight())
    {
        if (cooldownTimer >= attackCooldown)
        {
            cooldownTimer = 0;
            anim.SetTrigger("rangedAttack");
        }
    }

    if (enemyPatrol != null)
    {
        enemyPatrol.enabled = !PlayerInSight();
    }
}

// references
private void RangedAttack()
{
    SoundManager.Instance.PlaySound(FireballSound);
    cooldownTimer = 0;
    fireballs[0].transform.position = firepoint.position;
    fireballs[0].GetComponent<EnemyProjectile>().ActivateProjectile();
}

// references
private int FindFireball()
{
    for (int i = 0; i < fireballs.Length; i++)
    {
        if (!fireballs[i].activeInHierarchy)
        {
            return i;
        }
    }
    return 0;
}

// references
private bool PlayerInSight()
{
    RaycastHit2D hit = Physics2D.BoxCast(boxCollider.bounds.center + transform.right * range * transform.localScale.x * colliderDistance,
        new Vector3(boxCollider.bounds.size.x * range, boxCollider.bounds.size.y, boxCollider.bounds.size.z),
        0, Vector2.left, 0, playerLayer);

    return hit.collider != null;
}

// Unity Message | 0 references
private void OnDrawGizmos()
{
    Gizmos.color = Color.red;
    Gizmos.DrawWireCube(boxCollider.bounds.center + transform.right * range * transform.localScale.x * colliderDistance,
        new Vector3(boxCollider.bounds.size.x * range, boxCollider.bounds.size.y, boxCollider.bounds.size.z));
}

```

Slika 37: Delovanje metod RangedEnemy.cs

9.3 Glavni sovražnik (Šef (Boss))



Slika 38: Zgled glavnega sovražnika

Glavni sovražnik se pojavi samo v sceni Level2. Ustvarjen je bil tudi drugače kot prejšnja sovražnika. Igralca neprestano zasleduje in ga napade, ko je v njegovem dometu. Glavni

Sovražnik ima v sceni tudi drugo fazo imenovano **Enrage (se »razjezi«)** v, kateri postane hitrejši in hitreje napada. Preide v drugo fazo, ko izgubi dovolj življenjskih točk. Sestavljen je iz treh skript:

- Skripta **Boss.cs** je namenjena, da se obrača protu nekemu objektu (v igri je nastavljen proti igralcu),

```
public Transform player;

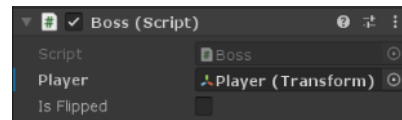
public bool isFlipped = false;

// Unity Message | 0 references
public void Start()
{
    Physics2D.IgnoreLayerCollision(8, 10, false); ;
}

// 1 reference
public void LookAtPlayer()
{
    Vector3 flipped = transform.localScale;
    flipped.z *= -1f;

    if (transform.position.x > player.position.x && isFlipped)
    {
        transform.localScale = flipped;
        transform.Rotate(0f, 180f, 0f);
        isFlipped = false;
    }
    else if (transform.position.x < player.position.x && !isFlipped)
    {
        transform.localScale = flipped;
        transform.Rotate(0f, 180f, 0f);
        isFlipped = true;
    }
}
```

Slika 40: Skripta Boss.cs



Slika 39: Vrednosti spremenljivk Boss.cs

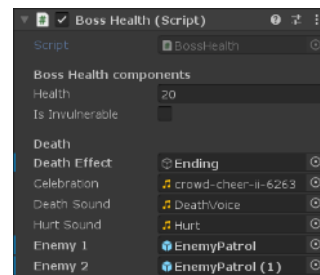
- Skripta **BossHealth.cs** se uporablja za obvladovanje zdravja in vedenja glavnega sovražnika (bossa) v igri. Določa obnašanje glavnega sovražnika, ob njegovi smrti in ob razjezitvi.

```
[Header ("Boss Health components")]
public int health = 20;
public bool isInvulnerable = false;
private SpriteRenderer spriteRend;

[Header("Death")]
public GameObject deathEffect;
[SerializeField] private AudioClip celebration;
[SerializeField] private AudioClip deathSound;
[SerializeField] private AudioClip hurtSound;
[SerializeField] private GameObject enemy1;
[SerializeField] private GameObject enemy2;

private bool isDead = false;
```

Slika 42: Deklaracija spremenljivk BossHealth.cs



Slika 41: Vrednosti spremenljivk BossHealth.cs

```

void Start()
{
    spriteRender = GetComponent<SpriteRenderer>();
}

1 reference
public void BossTakeDamage(int damage)
{
    if (isInvulnerable)
        return;

    health -= damage;
    SoundManager.Instance.PlaySound(hurtSound);

    if (health <= 6)
    {
        spriteRender.color = Color.red;
        GetComponent<Animator>().SetBool("IsEnraged", true);
    }

    if (health == 6)
    {
        SoundManager.Instance.PlaySound(deathSound);
    }

    if (health <= 0 && !isDead)
    {
        isDead = true;
        Die();
    }
}

@ Unity Message | 0 references
protected void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.tag == "Fireball")
        BossTakeDamage(1);
}

1 reference
void Die()
{
    SoundManager.Instance.PlaySound(deathSound);
    enemy1.SetActive(false);
    enemy2.SetActive(false);
    deathEffect.SetActive(true);
    Destroy(gameObject);

    SoundManager.Instance.PlaySound(celebration);
}

```

Slika 43: Delovaje metod BossHealth.cs

- In v skripti **BossWeapon.cs** nastavimo domet in število odvzema življenjskih točk ob napadu glavnega sovražnika (v normalnem in razjezenem stanju).

```

public int attackDamage = 1;
public int enragedAttackDamage = 1;

public Vector3 attackOffset;
public float attackRange = 1f;
public LayerMask attackMask;

0 references
public void Attack()
{
    Vector3 pos = transform.position;
    pos += transform.right * attackOffset.x;
    pos += transform.up * attackOffset.y;

    Collider2D colInfo = Physics2D.OverlapCircle(pos, attackRange, attackMask);
    if (colInfo != null)
    {
        colInfo.GetComponent<Health>().TakeDamage(1);
    }
}

0 references
public void EnragedAttack()
{
    Vector3 pos = transform.position;
    pos += transform.right * attackOffset.x;
    pos += transform.up * attackOffset.y;

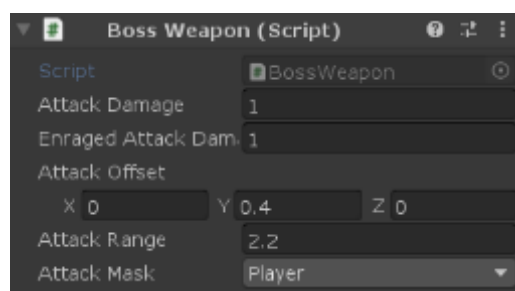
    Collider2D colInfo = Physics2D.OverlapCircle(pos, attackRange, attackMask);
    if (colInfo != null)
    {
        colInfo.GetComponent<Health>().TakeDamage(enragedAttackDamage);
    }
}

@ Unity Message | 0 references
void OnDrawGizmosSelected()
{
    Vector3 pos = transform.position;
    pos += transform.right * attackOffset.x;
    pos += transform.up * attackOffset.y;

    Gizmos.DrawWireSphere(pos, attackRange);
}

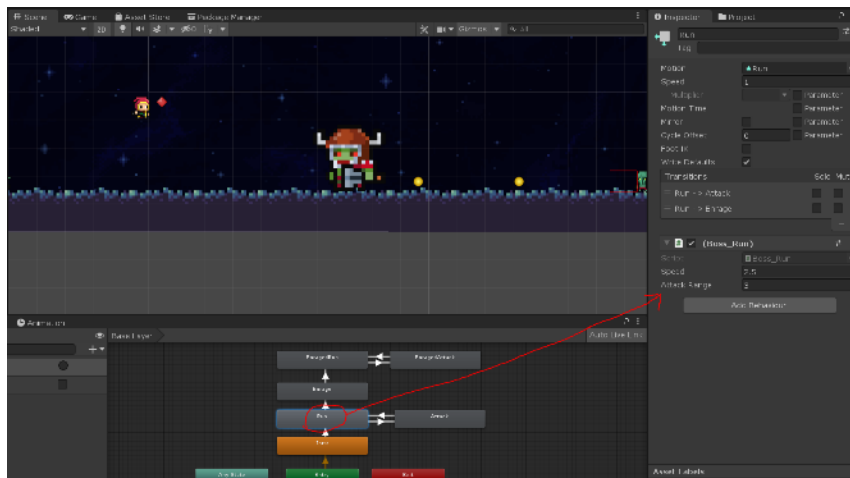
```

Slika 45: Skripta BossWeapon.cs



Slika 44: Vrednosti spremenljivk BossWeapon.cs

Skripte glavnega sovražnika se nahajajo tudi znotraj Animator komponente. Pod »Run« in »EnragedRun« animacijo ima dodeljeno **Boss_Run.cs** skripto in pod »Enrage« animacijo ima dodeljeno **Boss_Enrage.cs** skripto.



Slika 46: Prikaz dodeljene skripte Boss_Run.cs na Run animacijo

Skripta **Boss_Run** skripta določa premikanje glavnega sovražnika proti igralcu in izvajanje napada, če je igralec v določenem dometu.

```
public float speed = 2.5f;
public float attackRange = 3f;

Transform player;
Rigidbody2D rb;
Boss boss;

// Unity Message | 0 references
override public void OnStateEnter(Animator animator, AnimatorStateInfo stateInfo, int layerIndex)
{
    player = GameObject.FindGameObjectWithTag("Player").transform;
    rb = animator.GetComponent<Rigidbody2D>();
    boss = animator.GetComponent<Boss>();
}

// Unity Message | 0 references
override public void OnStateUpdate(Animator animator, AnimatorStateInfo stateInfo, int layerIndex)
{
    boss.LookAtPlayer();

    Vector2 target = new Vector2(player.position.x, rb.position.y);
    Vector2 newPos = Vector2.MoveTowards(rb.position, target, speed * Time.fixedDeltaTime);
    rb.MovePosition(newPos);

    if (Vector2.Distance(player.position, rb.position) <= attackRange)
    {
        animator.SetTrigger("Attack");
    }
}

// Unity Message | 0 references
override public void OnStateExit(Animator animator, AnimatorStateInfo stateInfo, int layerIndex)
{
    animator.ResetTrigger("Attack");
}
```

Slika 47: Skripta Boss_Run.cs

Skripta **Boss_Enrage.cs** se uporablja za aktiviranje neranljivosti glavnega sovražnika med animacijo **Enrage**.

```

@ Unity Message | 0 references
override public void OnStateEnter(Animator animator, AnimatorStateInfo stateInfo, int layerIndex)
{
    animator.GetComponent<BossHealth>().isInvulnerable = true;
}

@ Unity Message | 0 references
override public void OnStateUpdate(Animator animator, AnimatorStateInfo stateInfo, int layerIndex)
{
    ...
}

@ Unity Message | 0 references
override public void OnStateExit(Animator animator, AnimatorStateInfo stateInfo, int layerIndex)
{
    animator.GetComponent<BossHealth>().isInvulnerable = false;
}

```

Slika 48: Delovanje metod Boss_Enrage.cs

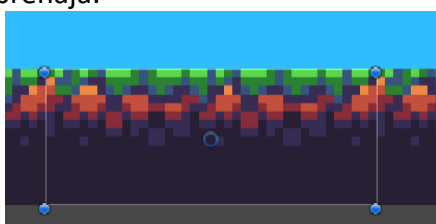
10. Zemljišče scen Level1 in Level2

V scenah Level1 in Level2 se nahajajo trije glavni zemljiščni objekti: tla, strop in stene. V sceni Level1 so dodani tudi okrasni objekti in vrata (imenujemo jih BossDoor) za prehod v sceno Level2.

V sceni Level1 je zemljišče razdeljeno na več sob z namenom omogočanja prehoda kamere iz ene sobe v drugo. Tako sem sprva načrtoval, vendar sem kasneje spremenil svojo odločitev in namesto tega nastavlil kamero, ki neprekinjeno sledi igralcu preko celotne scene, ne glede na prehode med sobami. Ta sprememba je bila narejena z namenom zagotavljanja boljše izkušnje igralca in enotne sledenje kamere med igro.

Tla

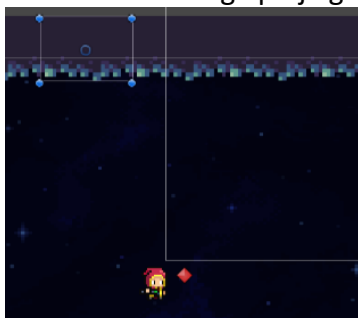
Po njih sprehaja se igralec sprehaja.



Slika 49: Zgled objekta tal

Strop

Objekt, ki preprečuje, da igralec ne uide iz vidnega polja glavne kamere.



Slika 50: Zgled objekta stropa

Stene

Igralec lahko skoči iz objekta sten.



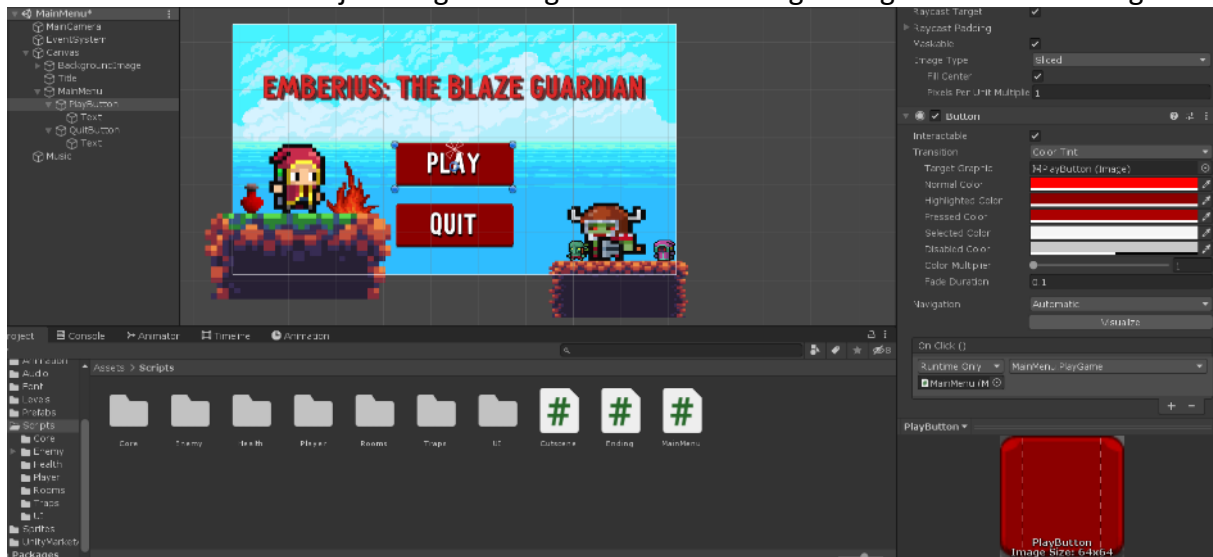
Slika 51: Zgled objekta stene

11. Meniji

V igri so trije meniji: Glavni meni, Meni konec igre in meni za premor.

11.1 Glavni meni

Glavni meni je samostojna scena, ki se prikaže ob zagonu igre, ob izstopu igralca iz igre ter ob uspešnem premagovanju igre. Meni vsebuje komponente uporabniškega vmesnika, ki jih ponuja Unity razvojno okolje. To so gumbi, besedilo, oblikovanje besedila, senčenje besedila itd. Meni vsebuje dva gumba: gumb za začetek igre in gumb za izhod iz igre.



Slika 52: Prikaz glavnega menija v urejevalniku

Meni vsebuje skripto v kateri sta metodi, ki določujeta funkcije gumbov. Metoda Playgame() je namenjena zamenjavi scene ob pritisku na »PLAY« gumb in metoda Quitgame() izstopi iz aplikacije ob pritisku na »QUIT« gumb.

```
public void PlayGame()
{
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex + 1);
}

0 references
public void QuitGame()
{
    Application.Quit();
}
```

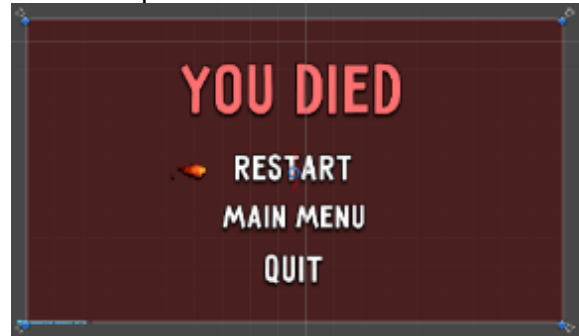
Slika 53: Delovanje metod MainMenu.cs

11.2 Meni konec igre in meni za premor

Menija enako delujeta, samo razlikujeta se po zgledu in način sklica. Meni konec igre se pojavi, če igralec nima več nastavljenih kontrolnih točk. Meni za premor se pa pojavi, ko igralec pritisne »ESC« v igri. Menija uporabljata enako skripto.



Slika 55: Zgled menija za premor



Slika 54: Zgled menija konec igre

Skripta **UIManager.cs** vsebuje tudi objekt **StartingInstructions**, ki se pojavi na začetku scene Level1 (ob zagonu igre). Namen objekta je obvestiti igralca o njegovih kontrolah.

Skripta UIManager.cs:

- **gameOverScreen (GameObject)**: Ta spremenljivka je referenca na igralni objekt (GameObject), ki predstavlja zaslon za konec igre. Spremenljivka se uporablja za prikaz ali skrivanje tega zaslona.
- **pauseScreen (GameObject)**: Ta spremenljivka je referenca na igralni objekt (GameObject), ki predstavlja zaslon za premor. Spremenljivka se uporablja za prikaz ali skrivanje tega zaslona.
- **startingInstructions (GameObject)**: Ta spremenljivka je referenca na igralni objekt (GameObject), ki predstavlja začetna navodila. Spremenljivka se uporablja za prikaz ali skrivanje teh navodil.
- **gameOverSound (AudioClip)**: Ta spremenljivka je zvočni posnetek, ki se predvaja ob prikazu zaslona za konec igre.

Metode:

- **Update()**: S pritiskom na tipko Escape se preverja, ali je treba prikazati ali skriti zaslon za premor. Če je zaslon za premor že prikazan, se odstrani, sicer pa se prikaže. V primeru, da je trenutna scena "Level1", se preverja čas, ki je potekel (v spremenljivki **lifetime**). Če je čas večji od 10 sekund, se izklopi prikaz začetnih navodil, sicer pa se prikažejo.
- **Awake()**: Ta metoda se kliče ob zagonu skripte. Uporablja se za skrivanje zaslona za konec igre.
- **Pause()**: Ta metoda se kliče ob pritisku na tipko Escape in se uporablja za premor igre. Če je zaslon za premor že prikazan, se odstrani in igra se nadaljuje z normalnim časovnim potekom. V nasprotnem primeru se prikaže zaslon za premor in igra se zamrzne.
- **GameOver()**: Ta metoda se kliče ob koncu igre in se uporablja za prikaz zaslona za konec igre. Aktivira se zaslon za konec igre in predvaja zvočni posnetek.
- **Restart()**: Ta metoda se kliče ob pritisku na gumb za ponovni zagon igre. Z uporabo **SceneManager.LoadScene()** se naloži trenutna scena, kar povzroči ponovni zagon igre.

- **MainMenu():** Ta metoda se kliče ob pritisku na gumb za vrnitev v glavni meni. Z uporabo **SceneManager.LoadScene()** se naloži prva scena igre (scena z indeksom 0), kar povzroči prehod nazaj v glavni meni.
- **Quit():** Ta metoda se kliče ob pritisku na gumb za izhod iz igre. Uporablja se za zapiranje aplikacije. Z uporabo **Application.Quit()** se zapre izvajanje aplikacije, kar privede do izhoda iz igre.

```

00 Unity Message | 0 references
private void Update()
{
    Scene currentScene = SceneManager.GetActiveScene();
    string sceneName = currentScene.name;

    if (Input.GetKeyDown(KeyCode.Escape))
    {
        Pause();
    }

    if (sceneName == "Level1")
    {
        lifetime += Time.deltaTime;
        if (lifetime > 10)
        {
            startingInstructions.SetActive(false);
        }
        else
        {
            startingInstructions.SetActive(true);
        }
    }
}

00 Unity Message | 0 references
private void Awake()
{
    gameOverScreen.SetActive(false);
}

1 reference
public void Pause()
{
    if (pauseScreen.activeInHierarchy)
    {
        pauseScreen.SetActive(false);
        Time.timeScale = 1f;
    }
    else
    {
        pauseScreen.SetActive(true);
        Time.timeScale = 0f;
    }
}

1 reference
public void GameOver()
{
    gameOverScreen.SetActive(true);
    SoundManager.Instance.PlaySound(gameOverSound);
}

0 references
public void Restart()
{
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
}

0 references
public void MainMenu()
{
    SceneManager.LoadScene(0);
}

0 references
public void Quit()
{
    Application.Quit();
}

```

Slika 56: Delovanje metod UIManager.cs

Puščica (ognjena krogla), ki se premika po meniju z puščico gor/dol ali w/s gumbi nam prikazuje trenutno izbrano opcijo na meniju.

Skripta SelectionArrow.cs (puščice):

- **options:** To je seznam (**RectTransform[]**), v katerem so shranjene pozicije Y-pozicij puščice za vsako možnost. Te pozicije se uporabljajo za premikanje puščice navzgor in navzdol med možnostmi.
- **rect:** To je komponenta **RectTransform**, ki je povezana s puščico za izbiro. Uporablja se za nastavljanje pozicije puščice.

- **currentPosition (int)**: To je trenutna pozicija puščice, ki se posodablja med premikanjem navzgor in navzdol.
- **changeSound (AudioClip)**: To je zvočni posnetek, ki se predvaja ob premikanju puščice navzgor ali navzdol.
- **interactSound (AudioClip)**: To je zvočni posnetek, ki se predvaja ob interakciji s trenutno izbrano možnostjo.

Metode:

- **ChangePosition**: Ta funkcija se kliče ob pritisku tipke za premikanje navzgor ali navzdol. Spreminja trenutno pozicijo puščice glede na podano smer. Preverja tudi, ali se je pozicija dejansko spremenila, predvaja zvok premikanja in nastavi ustrezno pozicijo puščice.
- **Interact**: Ta funkcija se kliče ob pritisku tipke "Enter" ali tipke "E" za interakcijo s trenutno izbrano možnostjo. Predvaja zvok interakcije in kliče funkcijo **onClick** gumba, ki je povezan s trenutno izbrano možnostjo.
- **Update** preverja se pritisk na tipke W, S, UpArrow in DownArrow, če je ena od teh tipk pritisnjena, se kliče metoda **ChangePosition** s pripadajočim argumentom (-1 za premik navzgor in 1 za premik navzdol). Preverja še pritisk na tipke "return" (enter) in E, če je ena od teh tipk pritisnjena, se kliče metoda **Interact**.

```

private void Awake()
{
    rect = GetComponent<RectTransform>();
}

// Unity Message / 0 references
private void Update()
{
    if (Input.GetKeyDown(KeyCode.W) || Input.GetKeyDown(KeyCode.UpArrow))
    {
        ChangePosition(-1);
    }
    else if (Input.GetKeyDown(KeyCode.S) || Input.GetKeyDown(KeyCode.DownArrow))
    {
        ChangePosition(1);
    }

    if (Input.GetKeyDown("return") || Input.GetKeyDown(KeyCode.E))
    {
        Interact();
    }
}

// 2 references
private void ChangePosition(int _change)
{
    currnetPosition += _change;

    if (_change != 0)
    {
        SoundManager.instance.PlaySound(changeSound);
    }

    if (currnetPosition < 0)
    {
        currnetPosition = options.Length - 1;
    }
    else if (currnetPosition > options.Length - 1)
    {
        currnetPosition = 0;
    }

    rect.position = new Vector3(rect.position.x, options[currnetPosition].position.y, 0);
}

// 1 reference
private void Interact()
{
    SoundManager.instance.PlaySound(interactSound);

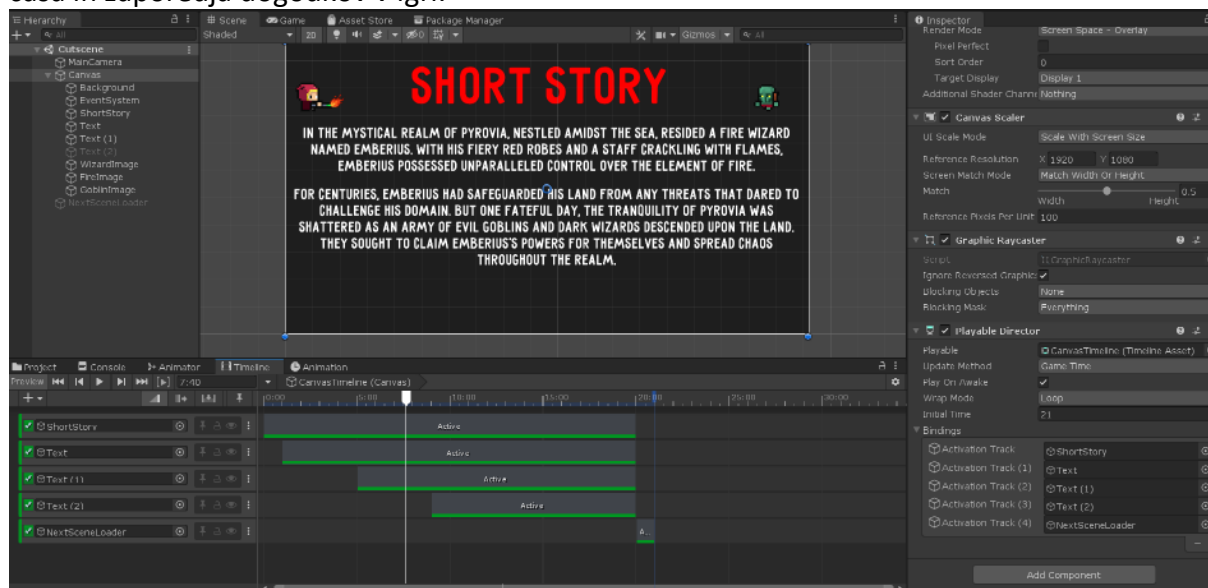
    options[currnetPosition].GetComponent<Button>().onClick.Invoke();
}

```

Slika 57: Delovanje metod SelectionArrow.cs

12. Posnetek

Scena **Cutscene (Posnetek)** uporablja orodje **Timeline**. Ta omogoča vizualno ustvarjanje in urejanje animacij ter dogodkov v igrah ali aplikacijah. Omogoča natančno nadzorovanje časa in zaporedja dogodkov v igri.



Slika 58: Zgled Posnetka in orodja Timeline v urejevalniku

V sceni posnetka se Timeline uporablja za vkapljanje objektov v določenem času. V prvih dvajsetih sekundah vkaplja deaktivirane tekstovne objekte. Na koncu posnetka pa vklopi objekt **NextScreenLoader**, katerega namen je prekop na sceno Level1.

Skripta, ki je vezana na NextScreenLoader objekt (Cutscene.cs):

```
private void OnEnable()
{
    SceneManager.LoadScene("Level1", LoadSceneMode.Single);
}
```

Slika 59: Delovanje metod Cutscene.cs

13. Zaključek

V zaključni nalogi sem se posvetil ustvarjanju 2D igre z naslovom »Emberius: The Blaze Guardian« v programu Unity. Igra je rezultat mojega trdega dela, učenja ter razvoja veščin programiranja. Skozi celoten proces sem se spopadal z različnimi izzivi, vendar sem se z zavzetostjo in vztrajnostjo uspel spoprijeti z njimi ter doseči svoje cilje.

V igro sem tudi želel vključiti funkcije, kot nadgradnje igralca (igralec izboljša obstoječe sposobnosti ali odklene dodatno opremo), dodatne mini igre, ki bi bile ločene od glavnega toka igre, vendar bi igralcem omogočile dodatno zabavo in izzive, več nivojev in izzivov z več vrst sovražnikov in pasti, dodatni zvoki (npr. pri teku)...

Izdelava igre je bila izjemna izkušnja, ki mi je omogočila razvoj veščin programiranja v C# ter spoznavanje pomembnih konceptov pri ustvarjanju video iger. Vesel sem dosežkov in rezultatov, ki sem jih dosegel z ustvarjanjem te igre, ter se veselim nadaljnjega razvoja in izpopolnjevanja svojih spretnosti v tem področju.

14. Viri in literatura

Učni material:

1. Unity User Manual. [Elektronski] 18. maj 2023. <https://docs.unity3d.com/Manual/index.html>.
2. Youtube. *Seznam predvajanja: Unity 2D Platformer for Complete Beginners*. [Elektronski] 18. maj 2023. <https://www.youtube.com/playlist?list=PLgOEwFbvGm5o8hayFB6skAfa8Z-mw4dPV>.
3. Youtube. *Video posnetek: How to make a BOSS in Unity!* [Elektronski] 18. maj 2023. <https://www.youtube.com/watch?v=AD4JIXQDw0s&t=95s>.
4. Youtube. *Video posnetek: START MENU in Unity*. [Elektronski] 18. maj 2023. https://www.youtube.com/watch?v=zc8ac_qUXQY&t=26s.
5. Youtube. *Video posnetek: [UNITY TUTORIAL] Add an Intro Story Cutscene to Your Game!* [Elektronski] 18. maj 2023. <https://www.youtube.com/watch?v=NnPDJfvLeWQ>.

Unity sredstva:

6. Unity Asset Store. *2D Pixel Item Asset Pack*. [Elektronski] 18. maj 2023. <https://assetstore.unity.com/packages/2d/gui/icons/2d-pixel-item-asset-pack-99645>.
7. Unity Asset Store. *Free 2D Mega Pack*. [Elektronski] 18. maj 2023. <https://assetstore.unity.com/packages/2d/free-2d-mega-pack-177430>.
8. Unity Asset Store. *Pixel Adventure 1*. [Elektronski] 18. maj 2023. <https://assetstore.unity.com/packages/2d/characters/pixel-adventure-1-155360>.
9. Unity Asset Store. *Pixel FX - Fire*. [Elektronski] 18. maj 2023. <https://assetstore.unity.com/packages/vfx/pixel-fx-fire-221146>.
10. Unity Asset Store. *Pixel Skies DEMO Background pack*. [Elektronski] 18. maj 2023. <https://assetstore.unity.com/packages/2d/environments/pixel-skies-demo-background-pack-226622>.
11. Unity Asset Store. *Pixel Heroes Animated*. [Elektronski] 18. maj 2023. <https://assetstore.unity.com/packages/2d/characters/pixel-heroes-animated-223662>.

Glasba:

12. Youtube. *Video posnetek (glasba): Welcome Space Traveler*. [Elektronski] 18. maj 2023. <https://www.youtube.com/watch?v=sfTuLuFXfko>.
13. Youtube. *Video posnetek (glasba): Jeremy Blake - Powerup! ♪ NO COPYRIGHT 8-bit Music*. [Elektronski] 18. maj 2023. <https://www.youtube.com/watch?v=l7SwiFWOQqM>.
14. Pixabay. *crowd cheering - DenisH18*. [Elektronski] 18. maj 2023. <https://pixabay.com/sound-effects/crowd-cheering-143103/>.

15. Zahvala mentorju

Na koncu bi se rad iskreno zahvalil svojemu mentorju, profesorju Tomu Kaminu, za njegovo nenehno podporo in nasvete med izdelavo zaključne naloge.

Izrekam tudi hvaležnost za potrpežljivo usmerjanje in pomoč pri premagovanju izzivov, s katerimi sem se soočal med delom na nalogi.

Prav tako bi se rad zahvalil za spodbudo in motivacijo, ki sem ju prejel od mentorja skozi celotno obdobje priprave zaključne naloge.

16. Izjava o avtorstvu

Spodaj podpisani dijak, Rok Rihar, izjavljam, da je ta zaključna naloga moje avtorsko delo.

Prav tako potrjujem, da nobena druga oseba, razen mentorja ni prispevala k tej nalogi. V celoti prevzemam odgovornost za vsebino te zaključne naloge.